

MLFF-DATA9A9a Restartable Production DATA6 Model Sweep

mdstats project

2026-07-30

1. Purpose

MLFF-DATA9A9a closes the execution-control gap between a qualified DATA5 partition and checkpoint-bound DATA6 model evidence. The scientific feature and prediction definitions already exist. This stage makes their expensive production realization:

- exact rather than ad hoc;
- restartable after interruption;
- tamper-evident at per-frame granularity;
- safe against sealed-role leakage;
- reusable by DATA6 without repeating foundation-model inference.

The stage does not fit DATA7 metrics, select training frames, write DATA8 jobs, train MACE, compare checkpoints, or open locked-test labels.

2. Ownership boundary

`mdstats.training_data.production_model_sweep` owns:

- deriving the exact DATA5-authorized descriptor/prediction frame union;
- per-frame descriptor and prediction artifact persistence;
- checkpoint and resume semantics;
- file/content digest verification;
- failure evidence;
- construction of a reusable `MaceDescriptorManifest` and `AtomicModelPredictionManifest`.

`mdstats.training_data.model_features` continues to own MACE descriptor and prediction semantics. `difficulty` owns residual and blinded summary records. `data6_bundle` only binds the verified sweep to DATA6. DATA7 and DATA8 remain separate stages.

3. Exact frame plan

`Data6ModelSweepPlan` is derived from:

- DATA3 frame-catalog identity;
- DATA5 partition identity;
- DATA6 policy identity;
- foundation-checkpoint identity;
- MACE descriptor-policy identity.

The descriptor set is the union of frames in the DATA6-declared descriptor roles. The prediction set is the union of:

- final-development training-difficulty frames;
- fold-training difficulty frames;
- materialized outer-monitor predictions;
- materialized calibration predictions;
- fold checkpoint-monitor predictions;
- fold evaluation predictions.

The locked interpolation test, purged frames, excluded frames, and any other frame outside the authorized union are recorded as sealed or excluded. The invariant is

$$\mathcal{U}_{\text{request}} = \mathcal{U}_{\text{descriptor}} \cup \mathcal{U}_{\text{prediction}}, \quad \mathcal{U}_{\text{request}} \cap \mathcal{U}_{\text{sealed}} = \emptyset.$$

A restored checkpoint whose plan digest differs from the newly derived plan fails before any model execution.

4. Per-frame artifacts

4.1 Descriptor sidecar

Each requested descriptor is written as

`descriptors/<frame_uid>.npy`

and records:

- frame and frame-record identities;
- checkpoint and descriptor-policy identities;
- shape and dtype;
- file SHA-256;
- canonical array-content digest.

4.2 Prediction sidecar

Each requested prediction is written as

`predictions/<frame_uid>.npz`

with energy, forces, and optional full 3×3 stress. The immutable record contains:

- frame and checkpoint identities;
- force shape and dtype;
- stress-presence flag;
- file SHA-256;
- separate canonical content digests for energy, forces, and stress.

The sidecar reader verifies every recorded property before returning an `AtomicModelPrediction`.

5. Restart and failure semantics

`Data6ModelSweepCheckpoint` stores the exact plan and all completed per-frame records. It has three states:

- `incomplete`: a valid resumable prefix/subset exists;
- `complete`: every frame has every artifact required by the plan;
- `failed`: a model or I/O failure occurred and the failing frame, exception type, and message are recorded.

NumPy sidecars are promoted through atomic file replacement. Each completed frame then appends one self-validating event to `data6_model_sweep_records.jsonl`. The journal begins with the plan digest, so a different DATA5/DATA6/checkpoint plan fails before reuse. A truncated final line from abrupt termination is discarded; all earlier newline-terminated events remain recoverable.

The complete `data6_model_sweep_checkpoint.json` is compacted only when an invocation returns normally or records a failure. The execution policy's `checkpoint_interval` controls journal flush and `fsync` cadence rather than rewriting the full history. The policy may also limit newly evaluated frames in one invocation; neither runtime control alters scientific plan identity.

On resume, each existing artifact is checked for:

- required-artifact completeness;
- file presence;
- SHA-256 identity;
- dtype and shape;
- finite numerical values;
- canonical array-content digest.

A corrupt or incomplete frame is recomputed when explicitly permitted. Otherwise resume fails closed. Verified frames are never recomputed.

6. DATA6 integration

DATA6 schema v5 may bind:

- the complete model-sweep plan;
- the descriptor manifest;
- the prediction manifest;
- the completed sweep-checkpoint digest.

The plan, checkpoint, descriptor manifest, prediction manifest, DATA6 policy, DATA5 bundle, frame catalog, and checkpoint identity must all agree.

A `PersistentAtomicModelPredictionCache` lazily reads and verifies prediction sidecars. DATA6 difficulty and blinded-prediction builders consume this cache, so a completed production sweep does not execute the foundation model again.

Historical DATA6 v1-v4 bundles remain readable. They receive no fabricated sweep identity.

7. Determinism and relocation

- Frame order is sorted by immutable frame UID.
- Sidecar names are relative and content-addressed by frame identity.
- Absolute output-directory location is execution metadata, not scientific content identity.
- JSON is canonicalized through the package digest contract.
- Restart boundaries do not change final manifests or DATA6 content digests.

8. Resource, scaling, and progress policy

Steady-state bookkeeping is amortized linear in newly processed frames:

- descriptor/prediction role membership uses hash sets;
- each completed frame writes its own sidecars and one append-only journal event;
- the growing plan and completed-record history are not reserialized per frame;
- one final compaction pass writes the canonical checkpoint and manifests.

The former full-checkpoint-per-interval design rewrote all completed records and all plan UID lists repeatedly, producing superlinear runtime and progressively slower frames on large campaigns. The append-only journal removes that growth. With `checkpoint_interval = 128`, an abrupt machine-level loss may require at most the latest 127 unflushed records to be recomputed; setting the value to 1 provides per-frame durable flushing at higher constant filesystem cost.

Progress rates are measured from newly processed frames in the current invocation. Restored frames are reported separately and are not divided by the new invocation's elapsed time. The displayed ETA uses a smoothed recent rate and also reports the current-invocation average.

The execution record does not claim asynchronous execution or distributed scheduling. Higher-level job-array orchestration may call the same plan in bounded runs, but concurrent writers to one sweep directory are not supported in this stage.

9. Required tests

The stage gate requires:

1. exact plan derivation and sealed-role exclusion;
2. plan, record, manifest, checkpoint, and execution-policy round trips;
3. interruption after a bounded number of frames;
4. verified resume without repeated inference;
5. persisted failure evidence and healthy-process resume;
6. descriptor and prediction corruption detection;
7. optional automatic recomputation of corrupt frames;
8. fail-closed plan/checkpoint mismatch;

9. DATA6 consumption without repeated descriptor or prediction calls;
10. historical DATA6 bundle compatibility;
11. one genuine MPA-0/LTA descriptor, energy, force, and stress smoke;
12. source/wheel export and installed-artifact smoke;
13. recovery from the append journal when the compact checkpoint is absent;
14. truncated-tail repair and subsequent restart;
15. one full checkpoint compaction per invocation, independent of frame count.

10. Completion and next boundary

DATA9A9a is complete when the checkpoint-bound model sweep can be interrupted, restored, verified, and consumed by DATA6 without re-inference. It does not imply that the complete 2,734-frame production sweep has finished.

The next stage, DATA9A9b, owns:

- execution of the complete production sweep;
- final and fold-local DATA7 materialization;
- exact replay-corpus binding;
- final and fold DATA8 job generation;
- a revised production qualification record.

Only after DATA9A9b passes may DATA9B protocol-matched training and freeze begin.